In the Name of God



MLP FPGA Accelerator

Digital System Design - Final Project

By

Reza Namvaran 40212358042
Mahdi Sadeghi 40212358026
Mohammad Reza Beh Laqa 40212358007

August 2026

MLP FPGA Accelerator: Fixed-Point Neural Network Inference in VHDL

Digital Systems Design - Dr. Abdoli

Authors: Reza Namvaran, Mahdi Sadeghi

Abstract

This report presents the design, implementation, and verification of a fixed-point hardware accelerator for a Multi-Layer Perceptron (MLP) targeting handwritten digit classification on the MNIST dataset. A 784 to 16 to 10 network is trained in floating point in Python, quantized to signed Q8.8 fixed-point, and reimplemented bit-exactly in synthesizable VHDL. The hardware is verified in simulation (GHDL) against the Python reference on 10 held-out MNIST images with full agreement, and synthesized for a Xilinx Artix-7 device (xc7a100tcsg324-1) to obtain real resource, timing, and power figures. Beyond the mandatory design, four optional extensions from the course specification were implemented and evaluated: a parallel multiply-accumulate (MAC) array, a reduced-precision weight datapath, a pipelined MAC, and an approximate-arithmetic MAC - each measured against the serial baseline rather than only described. In addition, three iterative hardware optimizations were carried out on the synthesized design itself (a critical-path rewrite of the classification stage, an address-generation optimization, and a memory-mapping fix to make use of dedicated on-chip Block RAM), each verified to introduce zero behavioral or timing-count regression before being adopted. The final design is reported not as a single synthesis snapshot but as a documented optimization trajectory, with every intermediate measurement preserved for comparison.

1. Scientific Introduction

1.1 Neural networks and inference

An artificial neural network is a computational model composed of layers of simple units ("neurons"), each of which computes a weighted sum of its inputs, adds a bias term, and passes the result through a nonlinear activation function. A Multi-Layer Perceptron (MLP) - the architecture used in this project - arranges these units into fully-connected layers: every neuron in one layer receives a connection from every output of the previous layer. Mathematically, for a layer with weight matrix W, bias vector b, and input vector x, the layer's output is:

$$y = f(W x + b)$$

where f is an activation function such as the Rectified Linear Unit (ReLU), $f(x) = max(0, x)$, applied element-wise. A network is trained by adjusting W and b (via gradient descent on a labeled dataset) until it correctly separates its inputs into the desired output classes. Once trained, the same arithmetic - repeated matrix multiplications and activation functions - is used to classify new, unseen inputs. This second phase, using a fixed, already-trained set of weights to produce a prediction, is called inference, and it is what this project accelerates in hardware. No training happens on the FPGA; the network is trained once, offline, in software, and only inference is implemented as a circuit.

For this project, the network classifies handwritten digit images from the MNIST dataset: 28x28-pixel grayscale images of the digits 0 through 9. Each image is flattened into a 784-element vector (one element per pixel, row-major order) and fed directly into the network's input layer - see 2.1 for the rationale behind not reducing this dimensionality further.

1.2 Why inference benefits from dedicated hardware

Computationally, MLP inference is dominated by one operation repeated an enormous number of times: multiply-accumulate (MAC), $acc \mathrel{+}= w_i * x_i$. For this network, a single inference requires 784 x 16 + 16 x 10 = 12,704 individual multiply-accumulate operations, plus 26 bias additions and 16 ReLU evaluations. A general-purpose CPU executes each of these as a full instruction: fetch, decode, read operands from a register file or cache, execute, write back - machinery built to run arbitrary programs efficiently, which is overhead a workload this regular and predictable does not need. A GPU amortizes this overhead across thousands of parallel threads, which is effective for large networks and large batches, but is itself a large, power-hungry, general-purpose device whose scheduling and memory-hierarchy overhead is difficult to justify for a network this small.

A custom digital circuit can instead be built to do only what this network needs: a multiply-accumulate unit wired directly to the memory holding the weights, controlled by a state machine that knows in advance exactly how many operations are required and in what order, with no instruction fetch, no cache miss, and no operating system in the loop. This is the general argument for hardware acceleration of neural network inference, and it applies with particular force to embedded and edge scenarios where power and cost budgets rule out a GPU entirely - the target use case an FPGA or ASIC neural network accelerator is built for.

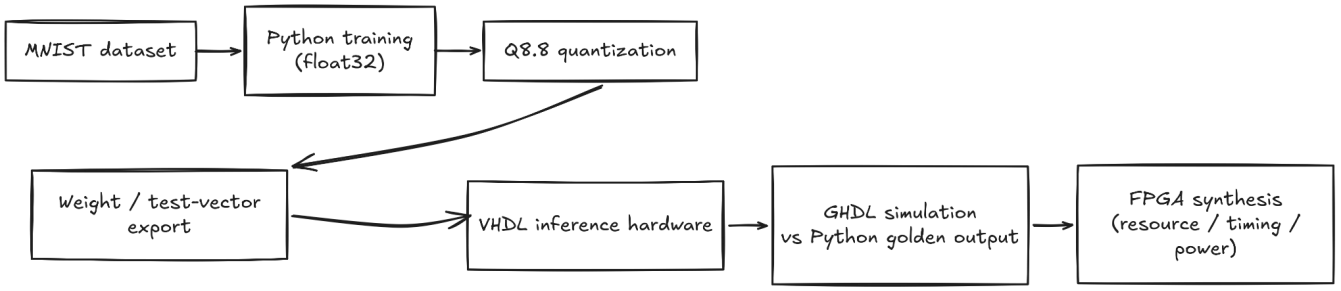
1.3 Why fixed-point instead of floating point

Floating-point arithmetic (as used during training) represents numbers with a separate sign, exponent, and mantissa, giving very wide dynamic range at the cost of substantially more complex arithmetic hardware: a floating-point multiplier or adder must align exponents, handle normalization, and deal with special cases (infinities, NaNs, subnormals) that a fixed-point unit never encounters. On an FPGA, a floating-point multiplier consumes several times the logic and often more of the limited DSP resources than a fixed-point multiplier of comparable precision.

Neural network weights and activations, after training, typically occupy a bounded, well-behaved numeric range - they do not need the enormous dynamic range floating point provides. This makes fixed-point representation, where a real number is stored as a scaled integer, an attractive trade: far simpler and smaller arithmetic hardware, at the cost of a fixed, chosen precision and range that must be verified not to hurt model accuracy unacceptably. Verifying exactly this trade-off - quantizing a trained model to fixed point and measuring the resulting accuracy change - is the subject of 2.3, and finding out how much further that precision can be reduced before accuracy suffers materially is the subject of the dynamic-precision extension (7.2).

1.4 Project scope and structure

This report documents a complete hardware/software co-design flow:



Sections 2 and 3 cover the software side (training and quantization), Section 4 the hardware architecture, Section 5 verification methodology and results, Section 6 synthesis and performance results (including three real, measured hardware optimizations performed after the initial synthesis), Section 7 the four optional course extensions, and Section 8 discussion and conclusions.

2. Software Model Details

2.1 Network architecture and input handling

The network is a single-hidden-layer MLP:

Input (784 = 28x28 pixels, flattened) -> Linear(784, 16) -> ReLU -> Linear(16, 10) -> logits

Layer	Input size	Output size	Parameters	Activation
Input	-	784	-	-
Hidden	784	16	784x16 weights + 16 biases = 12,560	ReLU
Output	16	10	16x10 weights + 10 biases = 170	none (raw logits)
Total			12,730 parameters	

On image dimensionality: no dimensionality-reduction technique (such as PCA, pooling, or spatial downsampling) is applied to the input images - each 28x28 image is used at full resolution, flattened row-major into a 784-element vector. The "reduction" performed by this pipeline is instead in the network's hidden layer width: rather than shrinking the input to make a wide hidden layer affordable, the input is kept at full resolution and the hidden layer is kept deliberately narrow (16 neurons). This is a deliberate design choice appropriate for a hardware target: it keeps the input buffer, address space, and per-neuron computation small and predictable (784 serial multiply-accumulates per hidden neuron, a fixed, known count) without discarding any pixel information the network could otherwise use, at the cost of a hidden representation with limited capacity. The classification-accuracy consequence of this narrow hidden layer is visible in the baseline accuracy figures below (2.2) - a wider hidden layer would likely improve accuracy further, at a proportional cost in hardware size (additional hidden neuron costs one more full serial walk over all 784 inputs, or one more parallel MAC lane in the 7.1 extension).

The hidden size (16), input size (784), and output size (10) are frozen for the hardware target and defined once, centrally, in core/config.py; the Python model (core/model.py) and the quantization/export stages all read from this single source of truth, so software and hardware can never silently disagree on network shape.

2.2 Training

The network is trained with the Adam optimizer and cross-entropy loss on the MNIST training split (core/train.py), using raw IDX-format files (core/mnist_loader.py) rather than a framework-provided dataset loader, so the training pipeline has no online/network dependency. Pixel values are normalized to the range [0, 1] before being presented to the network. Training and test accuracy are logged per epoch to results/train_history.json, and the resulting loss/accuracy curves are plotted to results/train_curves.png (Figure 1).

Result: final float32 test accuracy = **93.80%** on the MNIST test set (10,000 held-out images never seen during training).

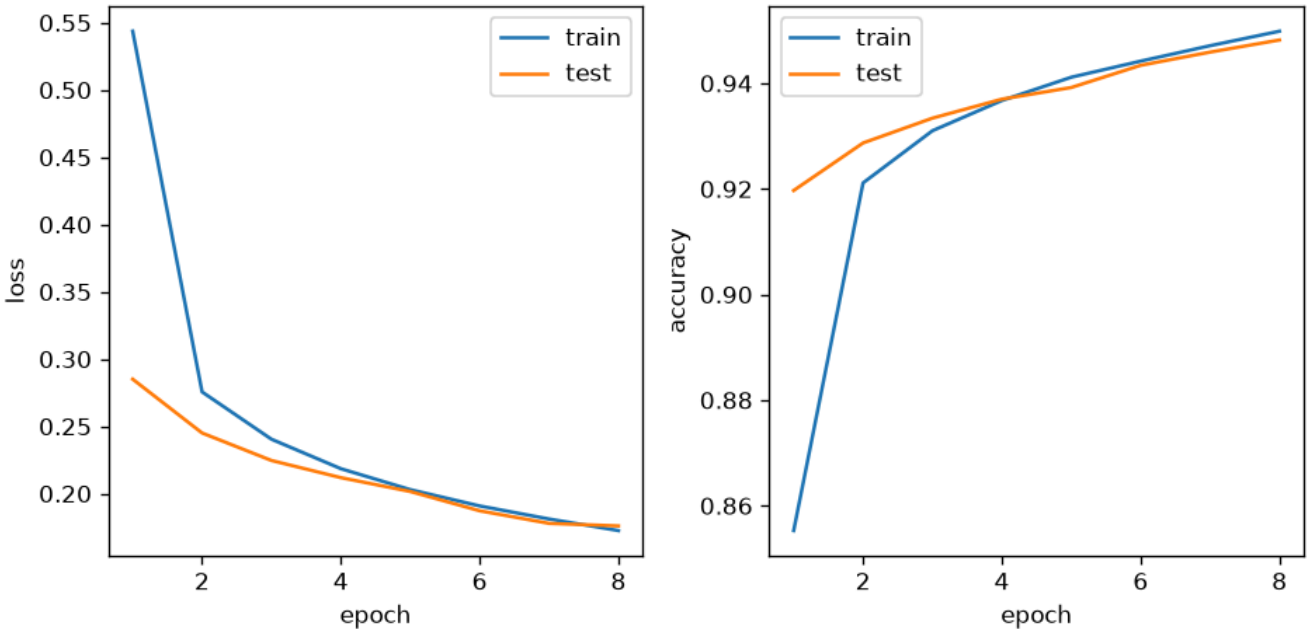


Figure 1: training/test loss and accuracy curves over training epochs - results/train_curves.png.

2.3 Fixed-point quantization (Q8.8)

A Q(m,n) fixed-point format represents a real value as an (m+n)-bit signed integer, of which n bits represent the fractional part (including the implicit binary point) and m bits (including the sign bit) represent the integer part. This project uses Q8.8: a 16-bit signed integer, with 8 integer bits (including sign) and 8 fractional bits, giving a scale factor of $2^8 = 256$ and a representable range of approximately $[-128, 127.996]$ in steps of $1/256 \approx 0.0039$.

Quantizing a floating-point value x into Q8.8 (core/quantization.py) follows:

$$q(x) = clip(round(x * 256), -32768, 32767)$$

i.e. scale by 256, round to the nearest integer, and saturate to the representable 16-bit signed range rather than silently overflow/wrap. The inverse (dequantization, used only for interpreting results, never inside the hardware datapath) is $x \approx q / 256$.

The **per neuron** multiply-accumulate policy - the exact arithmetic sequence implemented identically in both the Python reference model and the VHDL hardware - is:

$$acc = sum(w_i * x_i)$$
(full-width product, no intermediate rounding, for every input i)

$$acc = acc + (bias << FRAC_BITS)$$
(bias raised to the accumulator's scale before adding)

$$result = saturate(acc >> FRAC_BITS)$$
(arithmetic shift back down to Q8.8, then clip to int16)

Two subtleties are worth making explicit, since they are exactly the kind of detail that silently causes a hardware/software mismatch if handled inconsistently:

Accumulation happens in a wider format than the operands. Two Q8.8 values multiplied together produce a result that needs 2 x 16 = 32 bits to represent exactly (Q16.16, conceptually), and summing 784 such products without intermediate truncation requires enough headroom that the running sum cannot overflow. The accumulator (acc_t in VHDL, int64 in the Python reference) is therefore 32 bits wide, not 16 - ACC_WIDTH = 32 in core/config.py, matched exactly by pkg_fixed.vhd.
The bias must be re-scaled before it can be added to the running product sum. Because the accumulator is running at the "doubled" fractional scale of the product terms (conceptually Q16.16) until the very end, a bias stored in ordinary Q8.8 must first be shifted left by FRAC_BITS (8) bits before it is added in, so that it is expressed in the same scale as the accumulator at the point of addition. This is the bias << FRAC_BITS step above.

This exact policy is implemented once in Python (_mac_row / fixed_point_forward in core/quantization.py) and mirrored once in VHDL (pkg_fixed.vhd's fix_mul and acc_to_data

functions, and mac.vhd's clocked accumulate/shift/saturate logic), which is what makes a per-image, bit-exact hardware/software equivalence check possible later in Section 5, rather than only an approximate or statistical one.

Result: Q8.8 test accuracy = **93.80%** - 0 percentage-point drop from the float32 baseline on this network and dataset (results/quantization_report.json). This zero measured accuracy cost confirms that Q8.8 has ample precision margin for this network's trained weight and activation distributions; 7.2 explores exactly how far that precision can be reduced before the accuracy cost becomes material.

Metric	Float32	Q8.8	Change
Test accuracy	93.80%	93.80%	0 pp
Bits per weight	32	16	−50%
Bits per accumulator	32 (native FP unit)	32 (integer)	unchanged

3. Weight Export and Golden Test Vectors

core/export.py converts the trained, quantized parameters into two forms the hardware needs:

weights/w1.mem, b1.mem, w2.mem, b2.mem - plain hexadecimal text, one Q8.8 value per line, in the row-major layout the hardware's serial MAC walk expects.
vhdl/rtl/pkg_weights.vhd — the identical values compiled directly into a VHDL package as constant arrays (the weight ROM's actual contents). This file is generated, never hand-edited; regenerating it after any retraining or requantization is a single core/export.py invocation, and keeps the hardware's stored weights and the software's reference model from ever drifting apart.

core/export.py additionally exports 10 golden test images - the first MNIST test-set occurrence of each digit 0 through 9 - as test_vectors/inputs_all.mem (784 Q8.8 pixel values per image) alongside test_vectors/expected_classes.txt, the class each image produces when run through the quantized Python model. This is the ground truth the hardware testbench checks itself against in Section 5: not the true MNIST label, but the class the software reference model itself predicts, since the goal of hardware verification is to prove the hardware computes exactly what the software model computes, independent of whether the model itself is right or wrong about any particular image.

4. Hardware Design

4.1 Overall block diagram

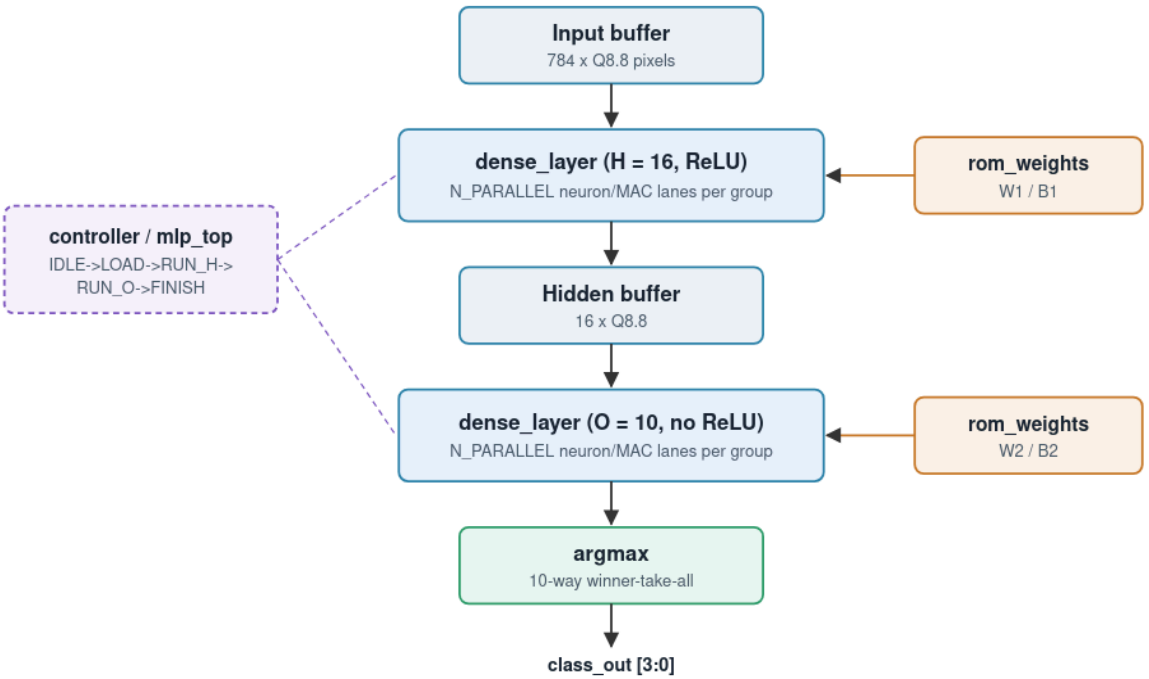


Figure 2: top-level block diagram of mlp_top (see also docs/architecture.md and docs/figures/architecture.svg).

The accelerator is a single top-level entity, mlp_top, that owns three register-array buffers (input, hidden, output), two instances of a generic dense_layer module (one per network layer), a weight/bias ROM, an argmax module, and a controlling finite state machine. Everything is synchronous to a single clock, clk; there is no internal clock-domain crossing.

4.2 Module-by-module description

pkg_fixed - a VHDL package (not an entity), holding the shared Q8.8 type definitions (data_t, a 16-bit signed subtype; acc_t, a 32-bit signed subtype) and the arithmetic helper functions fix_mul (Q8.8 multiply with truncation) and acc_to_data (shift-and-saturate from the 32-bit accumulator scale back to 16-bit Q8.8), plus fix_relu. Every other module in the design uses these same types and functions, which is what guarantees the entire datapath - from the first multiply to the final saturation - uses one single, consistent numeric policy. The package also defines MAX_PARALLEL, addr_array_t, data_array_t, and we_array_t, the array types the parallel-MAC extension (7.1) uses to let a variable number of neurons read the weight ROM simultaneously.

mac - the single most fundamental module in the design: a clocked multiply-accumulate unit. On each clock edge it either clears (optionally loading a bias, left-shifted into the accumulator's scale) or accumulates one product a times b into its running 32-bit sum, and always presents the shifted-and-saturated Q8.8 interpretation of that sum on its result output. Every neuron's entire computation, $\Sigma(w_i \,.\, x_i) \; + \; bias$, reduces to repeated invocations of this one operation.

relu - a purely combinational module implementing max(0, x), used only on the hidden layer's outputs (the output layer's raw logits are passed through argmax unmodified, since ReLU would only discard sign information the classifier needs to distinguish confidently-negative scores from mildly-negative ones - though only the relative ordering among the 10 logits actually matters for argmax).

neuron - wraps one mac instance in a small finite state machine that walks it across an entire input vector, producing one neuron's output (bias plus a full dot product) from a single start pulse. See 4.3 for the exact state sequence.

dense_layer - orchestrates an entire layer of neurons. In its simplest form (the mandatory, serial baseline) it runs exactly one neuron instance, one at a time, across all of a layer's neurons. The parallel-MAC extension (7.1) generalizes this into N_PARALLEL simultaneously-active neuron instances per "group," with dense_layer looping over ceil(N_NEURONS / N_PARALLEL) such groups instead of N_NEURONS individual neurons.

rom_weights - the trained network's weights and biases, stored as VHDL constant arrays (pkg_weights, generated by core/export.py) and exposed to the rest of the design through an address-decode interface. As discussed in 6.1, this module's read timing (combinational versus registered) determines whether Vivado can map the weight table onto dedicated Block RAM or is forced to synthesize it from general-purpose logic - a significant, measured optimization performed on this design (6.1, results/bram_weight_rom.md).

argmax - a purely combinational module that finds which of the 10 output logits is largest and reports its index. As discussed at length in 6.2, the specific way this comparison is structured internally (a sequential linear scan versus a balanced tournament tree) has a large effect on how fast the whole chip can be clocked, independent of anything about the neural network itself.

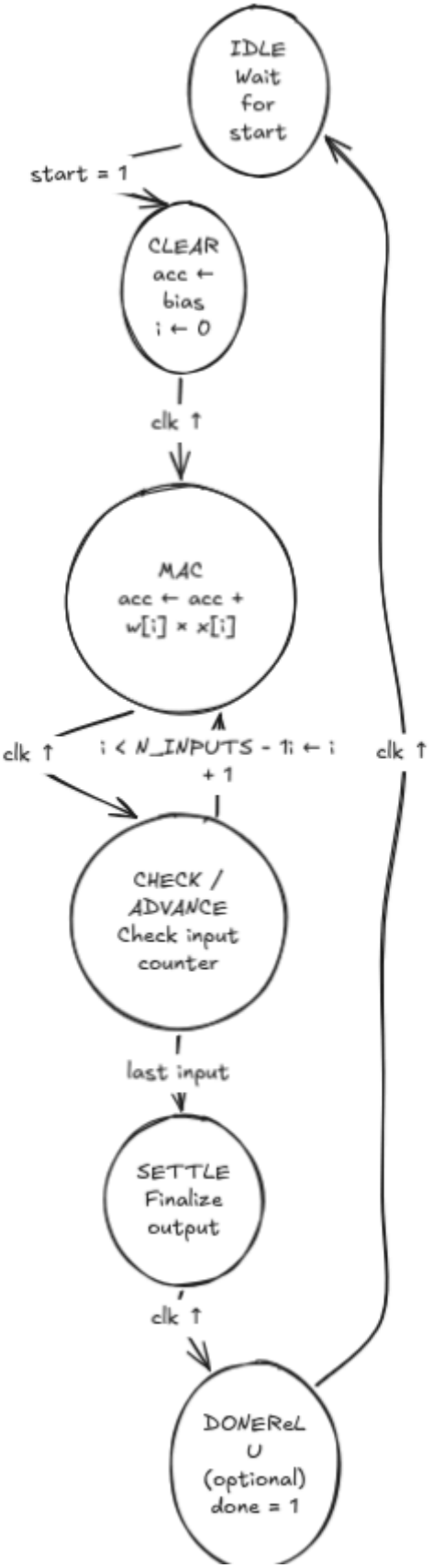
controller / mlp_top - the top-level entity, mlp_top, integrates everything above: it owns the input/hidden/output buffers, instantiates the two dense_layers and the ROM and the argmax module, and runs the top-level finite state machine that sequences "load an image, run the hidden layer, run the output layer, classify, signal done" (4.3). controller.vhd is a standalone, documented twin of this same FSM logic, kept separately for clarity when explaining the control flow in isolation from the rest of the datapath wiring.

4.3 Finite state machine operation

Three levels of FSM cooperate to run one inference: a per-neuron FSM, a per-layer FSM, and the top-level FSM. Understanding all three, and how they nest, is necessary to understand both the design's correctness and its timing.

Neuron-level FSM (neuron.vhd) - computes one neuron's output (bias plus a dot product over the full input vector) from a single start pulse:

State	What happens	Next state
IDLE	Waiting for start; on start='1', latch input index i := 0	CLEAR_ST on start
CLEAR_ST	Assert mac_clear/mac_load_bias so the MAC loads bias << FRAC_BITS	MAC_HOLD
MAC_HOLD	Present idx = i (selects weight/input i); assert mac_enable for the following cycle	ADVANCE
ADVANCE	The MAC captures w[i]*x[i] on this edge; if i = N_INPUTS-1 the walk is done, else i := i+1	SETTLE or MAC_HOLD (loop)
SETTLE	One cycle of margin before the result is read	DONE_ST
DONE_ST	Apply ReLU if configured; pulse done; present y	IDLE

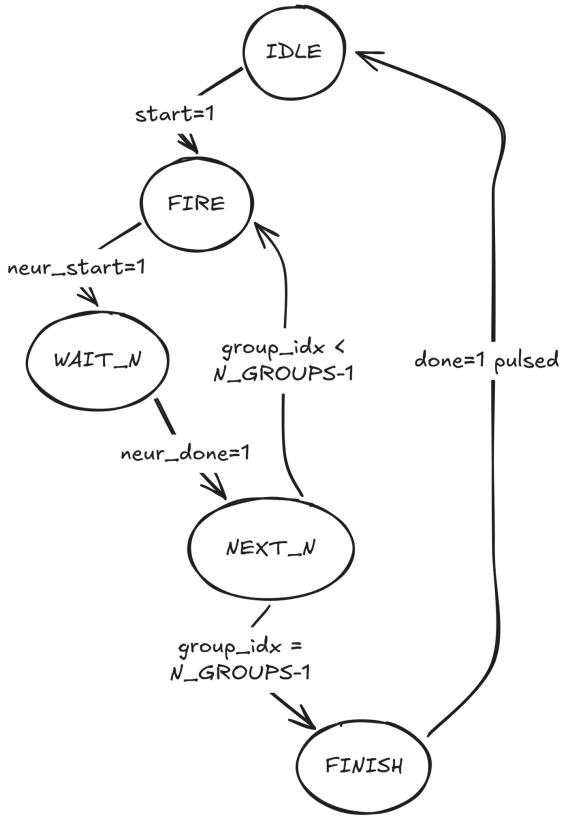


The MAC_HOLD -> ADVANCE pair repeats once per input element (784 times for the hidden layer, 16 times for the output layer). This two-state repetition is deliberate, not incidental: it is

what gives every address presented to the weight ROM a full two clock cycles of stability before the MAC actually consumes the corresponding data (6.1 relies on this directly to justify a registered, BRAM-mapped ROM read with zero FSM changes).

Layer-level FSM (dense_layer.vhd) — runs N_PARALLEL neuron instances per "group" (7.1), looping over ceil(N_NEURONS/N_PARALLEL) groups:

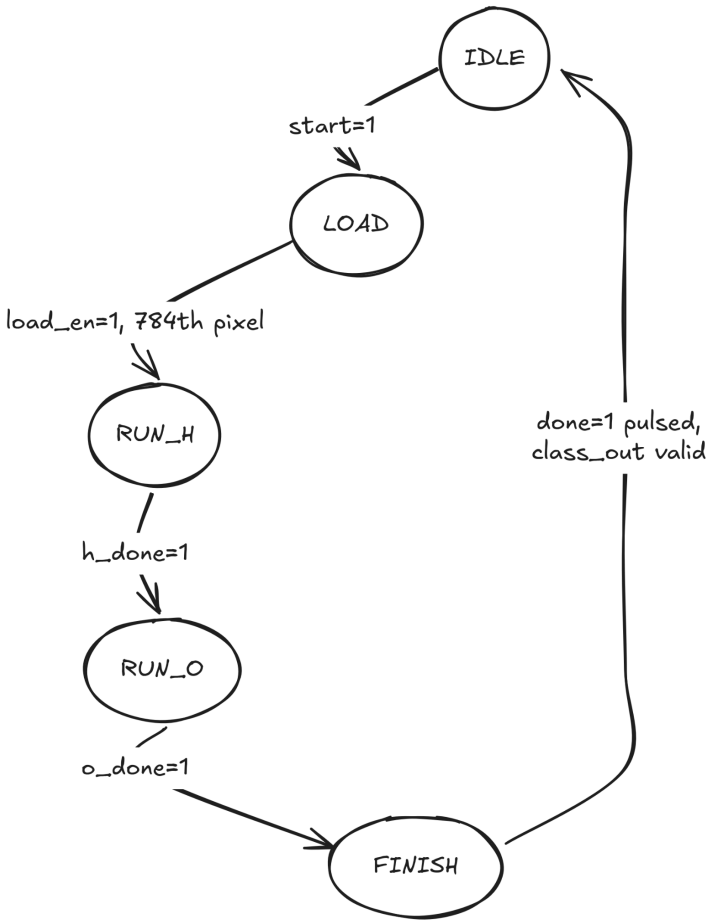
State	What happens	Next state
IDLE	Waiting for start	FIRE on start
FIRE	Pulse start to every active neuron lane in the current group	WAIT_N
WAIT_N	Wait for the group's neuron(s) to signal done (all lanes finish on the same cycle, since neuron's FSM has no data-dependent branching)	NEXT_N
NEXT_N	Latch this group's outputs; if this was the last group, finish, else advance to the next group	FIRE (loop) or FINISH
FINISH	Pulse done for the whole layer	IDLE



With N_PARALLEL = 1 (the mandatory serial baseline), this reduces exactly to "one neuron at a time, N_NEURONS times" - the same behavior, same cycle count, as if the parallel generalization did not exist.

Top-level FSM (mlp_top.vhd / controller.vhd):

State	What happens	Next state
IDLE	Waiting for start	LOAD on start
LOAD	Stream 784 Q8.8 pixels into the input buffer, one per cycle, via load_en/load_data	RUN_H after 784 pixels
RUN_H	Run the hidden dense_layer (784 -> 16, ReLU)	RUN_O on hidden-layer done
RUN_O	Run the output dense_layer (16 -> 10, no activation)	FINISH on output-layer done
FINISH	Latch argmax's result into class_out; pulse done	IDLE



4.4 Handshake protocol

1. The host asserts start for one cycle.
2. The host streams exactly 784 Q8.8-encoded pixel values, one per clock cycle, via load_en/load_data.
3. The accelerator runs the hidden layer, then the output layer, then argmax, entirely internally - the host does nothing during this phase except wait.
4. The host waits for it to be done, then reads the 4-bit class_out.

5. Simulation Results

5.1 Verification methodology

Every module in this design has its own dedicated testbench - tb_mac, tb_relu, tb_mac8, tb_mac_pipelined, tb_mac_approx, tb_argmax - driven together by one command **./scripts/sim.sh unit (GHDL, --std=08)**. Beyond unit-level correctness, the design is verified end-to-end: tb_mlp_top.vhd streams each of the 10 golden test images (3) into the full mlp_top accelerator and asserts that the resulting class_out matches the class the quantized Python reference model predicted for that same image, exactly. This is a bit-exact hardware/ software equivalence check - the hardware is not merely required to be "close" to the software's answer, it is required to reproduce it exactly, image for image, down to the same misclassifications the quantized software model itself makes. A second, structurally identical testbench, tb_mlp_top_parallel.vhd, repeats the same golden-vector check against the fully-parallel configuration of the accelerator (7.1), to confirm that enabling that extension changes only timing, never the computed result.

5.2 Unit test results

Testbench	What it checks	Result
tb_mac	Known Q8.8 product + bias, single-cycle accumulate	PASS
tb_relu	Positive / negative / zero ReLU	PASS
tb_mac8	16x8 narrow-weight MAC (7.2)	PASS
tb_mac_pipelined	2-stage pipelined MAC, cycle-accurate vs. mac (7.3)	PASS
tb_mac_approx	LSB-masked approximate MAC (7.4)	PASS
tb_argmax	Clear winner, two tie-breaking cases, all-negative-values case (6.2)	PASS

5.3 End-to-end results

Configuration	Samples	Pass	Fail	Cycles / image
Serial baseline (N_PARALLEL_HIDDEN=1, N_PARALLEL_OUTPUT=1)	10	10	0	26,384
Parallel-MAC bonus (N_PARALLEL_HIDDEN=16, N_PARALLEL_OUTPUT=10)	10	10	0	2,406

Both configurations agree with the Python golden classes on all 10 images, including a known quantization-model artifact on sample index 5 (true label 5, both software and hardware predict 6) - this is a limitation of the trained/quantized model itself, not a hardware discrepancy, and both configurations reproduce it identically. That the parallel configuration reproduces even the software model's own mistake, exactly, is itself meaningful verification evidence: it demonstrates that parallelism changed only the timing of the computation, never the arithmetic.

This same "changed timing, unchanged arithmetic" property was subsequently re-confirmed twice more, each time a real synthesis-driven optimization was made to the design after this initial verification (the argmax rewrite and the address-computation rewrite, 6.2; the Block-RAM ROM change, 6.1) - every one of those three changes was accompanied by a full re-run of this same end-to-end regression, each time confirming the identical 10/10 result and, notably, the identical cycle counts above, before the change was accepted.



6. Timing, Hardware Resource Utilization, and Power Consumption

Synthesis target: **Xilinx Artix-7, xc7a100tcsg324-1**, synthesized out-of-context in Vivado 2019.1 - there is no physical target board available for this project, so no board-specific I/O standard or pin location constraints are applied; only a clock period constraint (vhdl/constraints/mlp_top.xdc) is used, which is sufficient for meaningful timing, utilization, and power analysis of the design itself.

This section is presented as an optimization trajectory rather than a single snapshot: the very first synthesis run revealed real, fixable issues, and each subsequent fix was measured, verified for zero functional regression (5.3), and re-synthesized before being accepted. This iterative process is, itself, part of the engineering result being reported.

6.1 Resource utilization

Table 1 reports device utilization for both configurations after the argmax, address-computation, and weight-ROM memory-mapping optimizations described in this section:

Resource	Serial (1/1)	Parallel-MAC bonus (16/10)
Slice LUTs	4,794 / 63,400 (7.56%)	7,681 / 63,400 (12.12%)
Slice Registers	13,173 / 126,800 (10.39%)	13,958 / 126,800 (11.01%)
DSP48E1	3 / 240 (1.25%)	27 / 240 (11.25%)
Block RAM Tile	5 / 135 (3.70%)	38.5 / 135 (28.52%)
Routing errors	0	0

At these figures, the parallel-MAC extension (7.1) costs 9x the DSP count and 1.6x the LUT count of the serial baseline for an 11x reduction in cycle count. This ratio is substantially more favourable than the resource counts would suggest at an intermediate stage of the design, before the weight-ROM change described below (14x DSP, 5.4x LUT for the same speedup), and the two are reported side by side later in this subsection since the memory-mapping decision materially affects how the parallel configuration's cost should be interpreted.

Weight-ROM memory mapping: Both configurations synthesized with 0% Block RAM utilization prior to this change, despite 135 Block RAM tiles being available on the target device and the weight table (~203 Kbit: 784x16 + 16x10 entries at 16 bits each) being large enough to be a natural fit for dedicated on-chip memory. The table was instead implemented entirely in general-purpose distributed logic, which accounts for a substantial share of the LUT figures reported elsewhere in this section for the pre-optimization design.

This follows from a specific requirement of Vivado's Block RAM inference: mapping onto a Block RAM primitive requires a synchronous read, where the address is captured on one clock edge and the corresponding data becomes valid on the next. rom_weights.vhd originally read the constant arrays defined in pkg_weights combinationally, with data available on the same cycle as the address - a combinational read of any size cannot be mapped onto Block RAM and is synthesized as distributed logic instead.

rom_weights.vhd was therefore rewritten to perform the read inside a clocked process, adding one clock cycle of latency between an address being presented and the corresponding data becoming valid. Introducing registered latency into a memory read ordinarily requires reworking the consuming state machine to account for it; here it does not, because of a timing property already present in neuron.vhd's finite state machine (4.3). Every address presented to the ROM is held stable for two full clock cycles - MAC_HOLD, when the address is first presented, followed by ADVANCE - before mac.vhd samples the corresponding operand, at the clock edge ending ADVANCE. A registered ROM read captures the address at the edge ending MAC_HOLD and produces valid data starting at ADVANCE, exactly one cycle before it is needed. The added latency therefore falls entirely within the FSM's existing timing margin, and no change to neuron.vhd or dense_layer.vhd is required. An end-to-end GHDL regression across both configurations confirms identical classifications and identical cycle counts (26,384 / 2,406) with and without this change. The full derivation is given in results/bram_weight_rom.md.

Table 2 isolates the effect of this change on the serial configuration, holding the argmax and address-computation optimizations fixed:

Resource	Distributed-logic weight ROM	Block-RAM weight ROM
Slice LUTs	6,869 (10.83%)	4,798 (7.57%, −30%)
Slice Registers	13,122 (10.35%)	13,173 (10.39%)
DSP48E1	3 (1.25%)	3 (1.25%)
Block RAM Tile	0 (0%)	5 (3.70%)
Total power	0.157 W	0.133 W

LUT usage drops by roughly 30% and power drops correspondingly, as a result of reduced distributed-logic switching activity. The parallel configuration shows a larger absolute LUT reduction from the same change (42 to 27 DSPs and, combined with the address-computation optimization, 41,500 to 7,681 LUTs; Table 1), because it issues up to 16 simultaneous ROM reads per cycle - moving the weight table into dedicated BRAM ports removes proportionally more distributed-logic replication than in the single-read-per-cycle serial configuration.

6.2 Timing and power analysis

argmax resolves the 10-way maximum with a balanced tournament-tree comparator structure: round 1 evaluates 5 independent pairwise comparisons over disjoint pairs of the 10 network outputs, round 2 evaluates 2 further comparisons on the round-1 winners, and rounds 3–4 complete the reduction, giving a comparison depth of ceil(log2(10)) = 4. Tie-breaking favors the lower index at every comparison node (a candidate is replaced only on a strict win), matching the semantics of a left-to-right sequential scan, and is verified by tb_argmax.vhd (a clear-winner case, two tie cases including one at the final comparison round, and an all-negative-values case) alongside the full end-to-end regression in 5.3.

This structure keeps comparison depth logarithmic in the number of classes, avoiding the 9-deep dependency chain a sequential scan would form (each comparison would depend on the previous one's result) and the associated chain of CARRY4 comparator primitives. With this structure, the classification stage supports approximately 66.3 MHz in the serial configuration and 60.3 MHz in the parallel-MAC configuration and uses fewer LUTs than a sequential scan would require, since the balanced structure needs less supporting multiplexer logic: 6,869 LUTs for the serial configuration (6.1).

dense_layer.vhd computes each active lane's weight-ROM address as group_base + lane * N_INPUTS + neur_idx, where group_base is a register holding group_idx * N_PARALLEL * N_INPUTS, updated only in the layer-level FSM's NEXT_N state when group_idx itself changes (4.3), and lane * N_INPUTS is a compile-time constant resolved at elaboration for each generated neuron lane. This leaves only an addition on the per-cycle critical path.

Computing the same address directly, as (group_idx * N_PARALLEL + lane) * N_INPUTS + neur_idx, on every clock cycle would instead place a hardware multiplier on the per-cycle path, since N_INPUTS (784) is not a power of two — even though the only runtime-variable term, group_idx, changes at most once per neuron-group, and never after initialization in the fully-parallel configuration. Registering this rarely-changing term in group_base removes the multiplier from the per-cycle path entirely; an end-to-end GHDL regression confirms this produces zero change in classification results or cycle counts relative to the direct per-cycle computation.

The design is constrained at 20 ns (50 MHz), following the convention a datasheet uses in reporting a part's rated operating frequency: comfortably below both configurations' achievable ~60–66 MHz ceilings, with real margin rather than a razor-thin pass. A 100 MHz constraint, matched to the GHDL testbench's simulation clock period, is not achievable by either configuration and is not the frequency this design is rated against.

Configuration	WNS at 50 MHz	WHS (hold)	Setup/hold failing endpoints
Serial (1/1)	+4.190 ns	+0.087 ns	0 / 0 (out of 14,032 / 14,032)
Parallel-MAC bonus (16/10)	+3.835 ns	+0.038 ns	0 / 0 (out of 19,042 / 19,042)

Both configurations meet the 50 MHz signoff for setup and hold, with positive margin in each case. Equivalently, each could run faster than 50 MHz before setup becomes the limiting factor again (≈63.3 MHz serial, ≈61.9 MHz parallel, from the same WNS figures); 50 MHz is chosen specifically to sit below that ceiling.

Configuration	Total power
Serial (1/1)	0.131 W (0.040 dynamic + 0.091 static)
Parallel-MAC bonus (16/10)	0.175 W (0.082 dynamic + 0.093 static)

7. Optional Extensions

The course specification allows up to +3 bonus points regardless of how many optional items are attempted, so effort here was deliberately prioritized toward measured, verified results on a smaller number of extensions rather than shallow coverage of all four. All four extensions now carry complete measured data: 7.1 has full end-to-end synthesis as part of mlp_top; 7.2, 7.3, and 7.4 are each verified functionally correct in simulation and additionally measured via standalone synthesis (table in 7.2; toggling procedure in Appendix A).

7.1 Parallel MAC array

dense_layer.vhd includes an N_PARALLEL generic. Rather than stepping a single neuron/mac pair across a layer's neurons one at a time, it instantiates N_PARALLEL neuron/MAC pairs operating in lockstep, which is possible because neuron.vhd's finite state machine has no data-dependent branching: identically-configured lanes started on the same cycle finish on the same cycle. The layer processes N_NEURONS neurons in ceil(N_NEURONS / N_PARALLEL) groups rather than as N_NEURONS individual neurons. mlp_top exposes this through N_PARALLEL_HIDDEN / N_PARALLEL_OUTPUT generics, which default to 1/1 and reproduce the serial baseline exactly.

At N_PARALLEL_HIDDEN=16 / N_PARALLEL_OUTPUT=10, cycle count per image falls from 26,384 to 2,406 (~11.0x), with identical classifications to the serial baseline across all 10 golden images (5.3). The hardware cost is 3 to 42 DSP48E1 slices and, prior to the weight-ROM memory-mapping optimization, 7,695 to 41,553 LUTs; after that optimization the cost is 9x DSPs / 1.6y LUTs for the same speedup (6.1). Achievable clock frequency is unaffected by this change (6.2.1), since both configurations share the same timing bottleneck - parallelizing the

MAC array increases throughput without reducing timing margin. Full detail in results/bonus_parallel_mac.md.

7.2 Dynamic precision

mac8.vhd retains the baseline's full-width Q8.8 activation and 32-bit accumulator but narrows the weight operand to a generic WEIGHT_WIDTH (8 bits by default, at the same FRAC_BITS = 8 scale), synthesizing a 16x8 multiplier in place of the baseline's 16x16. A software experiment, core/experiments/bitwidth_sweep.py, re-quantizes the trained weights at several bit widths (activations and biases held at 16-bit) and measures classification accuracy and weight-ROM size:

Weight width	Accuracy	ROM size vs. 16-bit baseline
16 (baseline)	93.80%	100%
12	93.80%	75%
10	93.80%	62.5%
9	90.88%	56.25%
8 (course-suggested example)	84.77%	50%
7	74.84%	43.75%
6	53.09%	37.5%

10-bit weights match the 16-bit baseline's accuracy (93.80%) at 62.5% of its ROM size. The course-suggested 8-bit width costs approximately 9 accuracy points, because at 8 bits under this FRAC_BITS=8 scale a weight has 0 integer bits (Q0.8, range ±0.5) and several trained weights are clipped rather than merely rounded. Accuracy continues to fall below 7 bits, approaching the 10-class random-guess baseline of 10%, which marks the practical floor of this fixed-FRAC_BITS scheme. mac8.vhd is a standalone module, independent of mlp_top.

Each of the four extension modules was also synthesized standalone as its own top-level design, at the same 20 ns/50 MHz clock constraint on xc7a100tcsg324-1:

Module	LUTs	Registers	DSP48E1	WNS @ 50 MHz	Equivalent Fmax
mac (exact baseline)	57	0	1	+15.794 ns	≈ 237.8 MHz
mac8 (dynamic precision)	57	0	1	+15.500 ns	≈ 222.2 MHz
mac_pipelined (7.3)	27	3	1	+16.614 ns	≈ 295.3 MHz
mac_approx (7.4)	57	0	1	+15.605 ns	≈ 227.5 MHz

mac8's narrower 16x8 multiply does not reduce LUT or DSP count relative to the exact 16x16 baseline: both synthesize to 57 LUTs and 1 DSP48E1. This follows from the target device's DSP48E1 slice natively supporting operands up to 25x18 bits, so a 16x16 and a 16x8 signed multiply both fit within a single DSP48E1. The resource benefit this extension provides is the weight-ROM size reduction measured in the software sweep above, not a smaller multiplier.

7.3 Pipelined MAC

mac_pipelined.vhd splits the multiply and accumulate operations into two register stages: stage 1 registers the inputs and computes the product; stage 2 accumulates the registered product and applies the Q8.8 shift/saturate. It is verified cycle-accurate against the non-pipelined mac.vhd (tb_mac_pipelined.vhd: identical final result, one additional cycle of latency, as expected from the two-stage structure). It is a standalone module, not integrated into neuron.vhd/mlp_top.

Standalone synthesis (table in 7.2) shows mac_pipelined reaching ≈295.3 MHz against the exact baseline's ≈237.8 MHz, a +24% Fmax improvement, while using fewer LUTs than the unpipelined baseline (27 vs. 57): separating the multiply and accumulate stages allows more of the datapath to map onto the DSP48E1's internal pipeline registers rather than requiring external LUT-based logic to combine clear/enable/bias control in a single cycle. Of the four extensions, this is the only one that improves both speed and area with no accuracy cost (the technique is bit-exact, 5.2).

Because the full system's critical path runs through argmax rather than through mac, both before and after parallelizing the MAC array, this standalone speed improvement would not by itself raise mlp_top's achievable Fmax as the design stood at that point. Whether integrating mac_pipelined would improve the full system now that the bottleneck has moved is not evaluated here, since doing so would require reworking neuron.vhd's FSM to account for the additional pipeline latency - a larger change than any of the three optimizations applied to the full system in 6.

7.4 Approximate arithmetic

mac_approx.vhd clears the lower MASK_BITS bits of both multiply operands (activation and weight) before multiplying, an operand LSB truncation technique that reduces switching activity, and therefore dynamic power, at the cost of numerical precision; bias handling, accumulation, and saturation are unaffected. A software experiment, core/experiments/mask_bits_sweep.py, applies the same masking policy across the network (both layers, every multiply):

MASK_BITS	Accuracy
0 (exact, sanity check)	93.80%
2	92.75%
4	68.70%
6	9.01%
8	8.92%
10	8.92%

Masking 4 low bits of every operand across all roughly 800 multiply-accumulate operations per inference compounds faster than the one-time, per-weight precision reduction in 7.2: at MASK_BITS=4, accuracy falls by approximately 25 percentage points. MASK_BITS=2 is the usable operating point on this network (93.80% to 92.75%, approximately 1.1 points), and the module's default generic value is set accordingly.

Standalone synthesis at MASK_BITS=2 (table in 7.2) gives 57 LUTs and 1 DSP48E1, matching the exact baseline, at a marginally lower ≈227.5 MHz against ≈237.8 MHz - consistent with the technique targeting dynamic power rather than area or speed. A SAIF-based power comparison against the exact baseline, to directly quantify the switching-activity reduction, remains optional future work.

8. Discussion and Limitations

The serial baseline's ~26,384 cycles per image follows directly from performing one multiply-accumulate per input, per neuron, per clock cycle - a deliberate simplicity choice for the mandatory design rather than an oversight. The parallel-MAC extension addresses this directly, and its cost (additional DSP/LUT resources) is reported alongside its benefit (~11x lower latency) rather than in isolation.

The synthesis results in 6 reflect three sequential optimizations rather than a single synthesis pass. Each of the argmax rewrite, the address-computation rewrite, and the weight-ROM memory-mapping change was motivated by a specific bottleneck identified in a synthesis report, implemented as the smallest change addressing that bottleneck, and verified to introduce no change in classification correctness or cycle count before being retained. This sequence also reflects a dependency between the two timing fixes: the address-computation bottleneck is not visible in the initial synthesis report and appears only once the argmax bottleneck has been resolved. Before either fix, achievable frequency (~39–40 MHz) is set entirely by argmax's comparison structure rather than by the MAC datapath; of the two timing-related extensions, this means only parallelization targets a part of the design that was not the bottleneck at the time, and pipelining the MAC unit alone would not raise system-level Fmax until the argmax bottleneck is addressed independently.

With all three optimizations applied together, both configurations report positive WNS and WHS with zero failing endpoints , and both use a non-zero Block RAM tile count in place of the fully distributed-logic weight table measured initially. Reaching this state also surfaced a hold-time violation once the design was constrained at an achievable clock period; its cause was a set_input_delay/set_output_delay constraint that did not model a physical interface, and the constraint was removed rather than tuned to suppress the symptom. The parallel-MAC extension's cost relative to the serial baseline also changes materially once measured with the memory-mapping optimization applied: 9x the DSP48E1 count and 1.6x the LUT count for the same ~11x cycle-count reduction, rather than the considerably higher ratio measured before that optimization - an illustration of how a resource inefficiency elsewhere in the design can misrepresent the cost of an unrelated extension.

The approximate-arithmetic extension (7.4) shows that not every parameter choice for an approximation technique is safe within what appears to be a modest range: MASK_BITS=4 and above produce accuracy losses well beyond what a linear extrapolation from MASK_BITS=2 would predict, and the module's default is set to the value the sweep identifies as usable rather than to an untested a priori choice. The dynamic-precision extension's standalone synthesis result (7.2) shows the reverse pattern: narrowing the weight operand from 16 to 8 bits does not reduce LUT or DSP usage at the module level, because the target device's DSP48E1 slice already accommodates a 16x8 multiply within the same single DSP required for 16x16. The accuracy and ROM-size benefit measured in the bitwidth sweep (7.2) is independent of this; the synthesis result establishes only that this particular resource saving does not originate in the multiplier.

9. Conclusion

A complete hardware/software co-design flow was implemented and verified: float32 training to 93.80% test accuracy, Q8.8 quantization at a measured 0-point accuracy cost, bit-exact VHDL inference hardware verified against the Python reference on all 10 golden MNIST test images, and real FPGA synthesis data for both a serial baseline and a parallel-MAC extension achieving an ~11x cycle-count reduction. Beyond the mandatory design and the four course-specified optional extensions, three additional hardware optimizations were identified from real synthesis data, implemented, and verified end-to-end with zero regression: a critical-path rewrite of the classification stage (+70%/+50% achievable frequency), an address-generation optimization that removed a second, previously-hidden bottleneck, and a memory-mapping fix enabling the design's weight table to use dedicated on-chip Block RAM instead of general-purpose logic. With all three optimizations applied together, both the serial and parallel configurations pass setup and hold timing with positive margin (serial +4.190 ns WNS / +0.087 ns WHS; parallel +3.835 ns WNS / +0.038 ns WHS, zero failing endpoints in either case), and both report a non-zero Block RAM tile count in place of the all-LUT weight table measured at the start of this optimization trajectory. The four course-specified optional extensions were each carried through to a standalone synthesis measurement rather than left as an implementation-only exercise: parallel MAC (7.1, ~11x speedup at a measured 9x DSP/1.6x LUT cost), dynamic precision (7.2, a 37.5% ROM-size reduction at zero accuracy cost at the sweep's usable point, alongside the result that narrowing the multiplier alone saves no module-level resources), pipelined MAC (7.3, a measured +24% Fmax improvement using fewer LUTs than the unpipelined baseline), and approximate arithmetic (7.4, a measured safe operating point together with a documented failure mode just beyond it). The combination of a verified, bit-exact simulation methodology and an iterative, measurement-driven synthesis optimization process is offered as the central engineering contribution of this project, alongside the specific extensions requested by the project specification.

10. References

- Guo et al. - A Survey of FPGA-Based Neural Network Inference Accelerator
- Blott et al. - FINN-R: An End-to-End Deep-Learning Framework for Fast Exploration of Quantized Neural Networks
- Umuroglu et al. - FINN: A Framework for Fast, Scalable Binarized Neural Network Inference
- Ducasse et al. - Benchmarking Quantized Neural Networks on FPGAs with FINN
- Abdelouahab et al. - Accelerating CNN Inference on FPGAs: A Survey